SENG 533: Concrete Proposal

Group 14

Name	UCID
Oluwafisayo Adabonyan	30141541
Eshilama Akalumhe	30140722
Kameel Kasumu	30141921
Aly Mohamed	30139727

Introduction & System Definition Description
We will be evaluating a GPU-enabled deep learning pipeline on Google Colab

The system under evaluation is a GPU-enabled deep learning training and inference pipeline deployed on Google Colab. The system trains and evaluates convolutional neural networks for multi-class MRI brain tumour classification using a publicly available Brain Tumour MRI dataset (7023 labelled images across four classes: glioma, meningioma, pituitary, no tumour).

The implementation uses:
- TensorFlow
- Keras
- GPU acceleration (NVIDIA T4 or dedicated NVIDIA GPU on a personal laptop/pc)

Two model families are evaluated:
1. Custom CNN (trained from scratch)
2. Transfer learning using EfficientNet (B0 variant)

The workload includes:
- Image preprocessing
- Model training
- Validation
- Inference

Define Goals/Objectives
The project evaluates trade-offs between predictive performance and computational efficiency in a cloud-based GPU environment.

We aim to:
1. Quantify training and inference cost differences between the custom CNN classifier and EfficientNet based transfer learning classifier.
2. How do training time, inference latency, and throughput vary with workload intensity (e.g., batch size, epochs) and scalability factors on Colab?
3. Identify GPU utilization efficiency and system bottlenecks.
4. Compare statistics between the two models and determine which modeloffers the best balance between predictive accuracy and computational efficiency

Literature Review
Typically, studies related to evaluating deep learning performance in medical image classification focus primarily on predictive accuracy comparisons between different model architectures. Many works compare custom CNNs, ResNet variants, or EfficientNet-based transfer learning approaches and report classification metrics such as accuracy, precision, recall, and F1-score. While these comparisons help identify high-performing models, they often do not examine the system-level performance characteristics of the deep learning pipeline itself. In particular, limited attention is given to how workload intensity, batch size, fine-tuning depth, or GPU allocation impact training time, inference latency, memory usage, and GPU utilization. Furthermore, experiments conducted in cloud environments such as Google Colab rarely account for performance variability caused by shared GPU resources. There is a lack of in-depth evaluation of how specific factors directly influence computational cost and scalability. Understanding these relationships is essential for developers and researchers who must deploy efficient and reliable medical AI systems under resource constraints. Our focus is to perform a controlled and statistically grounded evaluation of these factors to better understand how they impact overall deep learning pipeline performance.

Metrics and Factors

Define Metrics

To evaluate the performance of the deep learning pipeline comprehensively. Both predictive and system-level metrics are required. They include:

- **Training Time**: measure the total wall-clock time required to complete the model training and the time per epoch.
- **Interference Latency**: measure the time required to generate predictions for a single image and for a batch of images.
- **Throughput**: evaluate the number of images processed per second during training and inference.
- **GPU Utilization**: measure the percentage of GPU compute resources used during execution.
- **GPU Memory Usage**: measure memory consumption (MB) and peak memory utilization under different workloads.
- **Predictive Accuracy**: evaluate classification accuracy and F1-score to ensure performance gains are not achieved at the expense of correctness
- **Statistical Reliability**: report mean, standard deviation, and 95% confidence interval across repeated runs.

Important Factors Identification

Factor	Levels
Model Architecture	Custom CNN, EfficientNet-B0
Batch Size	16, 32, 64
Number of Epochs	5, 10, 20
GPU Type	NVIDIA T4, Google TPU v5e-1, Dedicated GPU(s) on Laptop/PC
Image Resolution	Fixed baseline (e.g. 224 x 224)
Workload Intensity	Low (small batch + few epochs), Medium, high (large batch + more epochs)
Execution Repetition	Minimum 3 runs per configuration

These factors allow us to systematically analyze how architectural choices and workload scaling directly impact computational cost, scalability and resource efficiency within a cloud-based GPU environment.

Evaluation Strategy

We will use a controlled experimental benchmarking. Each configuration will be executed multiple times to account for performance variability in Google Colab's shared GPU environment.

Repetition Strategy

Each experiment is repeated at least 3 times, reporting the mean, standard deviation, and 95% confidence interval to ensure statistical reliability.

Evaluation Setup

We will use a fixed dataset split, fixed image resolution, Adam optimizer, identical preprocessing pipeline, and a fixed random seed, with GPU statistics monitored. Each run is executed in a fresh runtime with cleared GPU memory, and the GPU type is recorded.

Test Workload Design
Workloads are designed to stress:

1. Compute capacity (larger batch sizes)
2. Memory limits (high batch sizes + fine tuning)
3. Training duration scaling (increasing epochs)
4. Inference latency under real-time constraints

Experiment Design
To systematically evaluate the performance of the system, a factorial experimental design will be used to evaluate how key factors influence computational and predictive performance. The primary controllable factors are:

- Model architecture (Custom CNN, EfficientNet-B0)
- Batch size (16, 32, 64)
- Number of epochs (5, 10, 20)

This results in $2 \times 3 \times 3 = 18$ core configurations. Image resolution (224×224), optimizer (Adam), dataset split, preprocessing pipeline, and random seed are fixed to ensure comparability.

The experimental process is structured in three stages:

<u>1. Baseline Training Experiments</u>
All 18 configurations are executed to measure:

- Total training time and time per epoch
- GPU utilization and memory usage
- Training throughput
- Validation accuracy and F1-score

Workload intensity is implicitly defined by batch size and epoch combinations (low, medium, high).

<u>2. GPU Sensitivity Analysis</u>
To account for hardware variability, selected configurations will be compared across observed GPU types. This analysis evaluates whether performance differences are attributable to architecture or underlying hardware allocation. Each configuration will be executed using a fresh runtime, with the GPU type recorded for every run.

<u>3. Inference Benchmarking</u>
Standalone inference experiments will be conducted on the test set to measure:

- Single-image latency
- Batch inference latency
- Inference throughput

This isolates deployment performance from training cost and enables direct comparison between the custom CNN and EfficientNet-B0.

Implementation is performed within the existing PyTorch pipelines. Timing is measured using wall-clock time functions, and GPU statistics are collected using PyTorch CUDA utilities and system-level monitoring tools. All results are logged for structured post-processing.

Analysis of Results
Results will be analyzed using descriptive and inferential statistics to ensure reliability and interpretability.

For each metric, we will report:

- Mean

- Standard deviation
- 95% confidence interval (across all the runs)

Factor effects (e.g., batch size, architecture) will be evaluated using ANOVA or t-tests where appropriate (significance threshold $p < 0.05$). Simple regression analysis will be used to model trends such as training time or memory usage as a function of workload intensity.

Visualizations will include:
- Box plots for metric distributions across factor levels
- Line plots showing scalability trends
- Bar charts comparing the overall computational cost between models

Bottlenecks will be identified by examining relationships between GPU utilization, memory usage, runtime, and failure cases (e.g., out-of-memory errors). Where GPU allocation introduces variability, sensitivity analyses will be performed on subsets grouped by GPU type.

This design enables a statistically grounded comparison of the Custom CNN and EfficientNet-B0, allowing us to determine which model offers the best balance between predictive accuracy and computational efficiency in a constrained cloud GPU environment.

Timeline

Task	Duration
● Set up measurement instrumentation ● Modify the current code for the two models to align with our initial setup plan	February 23rd - February 28th
● Run initial pilot experiments ● Validate logging accuracy ● Adjust batch size ranges if needed	March 1st - March 6th
● Execute a full experimental matrix ● Collect GPU utilization logs ● Store results in structured CSV format	March 7th - March 21st
● Perform statistical analysis ● Generate graphs ● Identify bottlenecks	March 22nd - March 29th
● Draft results section ● Write a scalability and trade-off discussion	March 30th - April 5th
● Compile report ● Internal Review ● Final Report ● Final submission	April 6th - April 10th